CALIFORNIA STATE UNIVERSITY
LONG BEACH

CECS 341: Computer Architecture & Organization

Hossein Sayadi, PhD

Department of Computer Engineering & Computer Science
California State University, Long Beach

Fall 2025

Processor Pipelining

Lecture Outline

- What is the concept of pipelining?
- How does pipelining improve performance?
- Dividing the instruction cycle into stages.
  - Fetch, decode, and execute stages.
- The MIPS 5-stage pipeline.
  - Fetch, decode, computation, memory access, and write-back stages.

Introduction to Pipelining

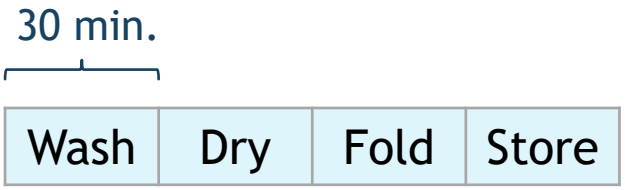
Analogy

- Pipelining a processor is similar to a laundry cycle or using an *assembly line* to build an automobile.

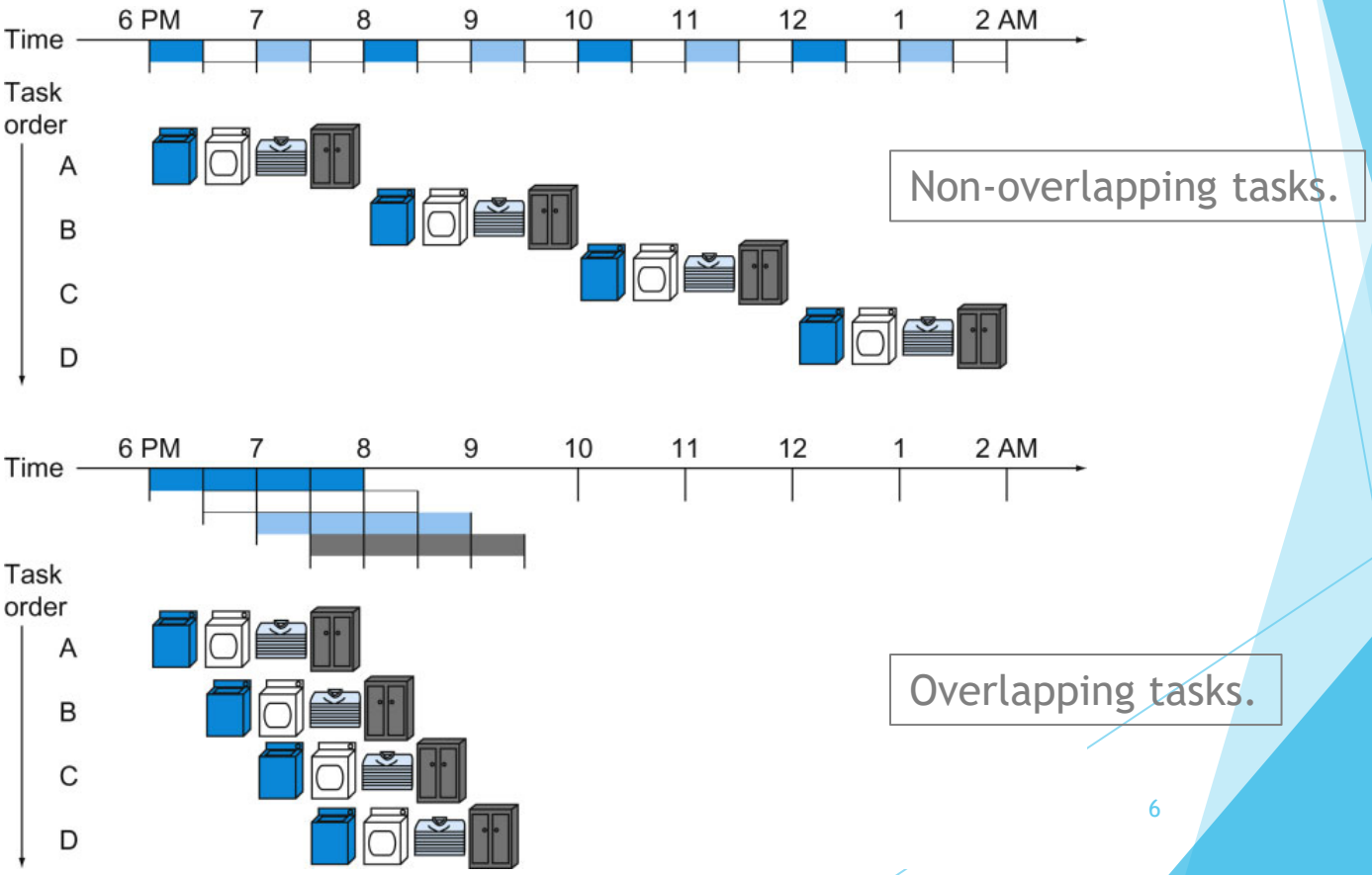
- Questions to address:
  - How long does it take for one worker to build a car?
  - How long will it take if the tasks are divided among four workers?
  - Can the tasks be more equally divided among the workers?
  - What if one stage of the assembly line takes longer than the others? Will the other workers remain idle or can they continue working?

Analogy

- Laundry Cycle

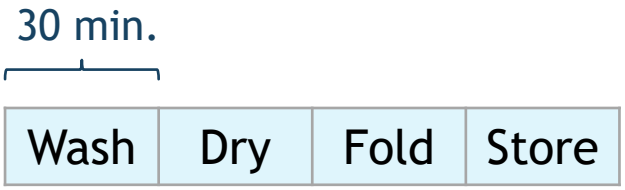


- Divided into 4 tasks.
- Each requires 30 minutes.

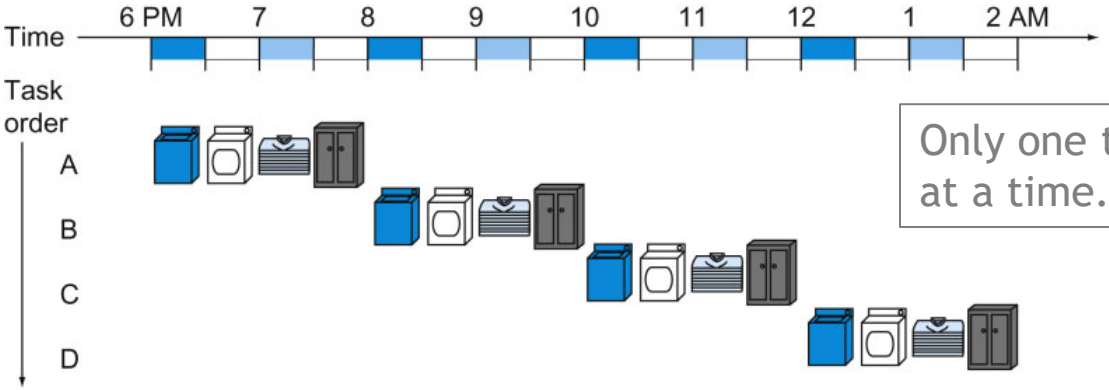


Analogy

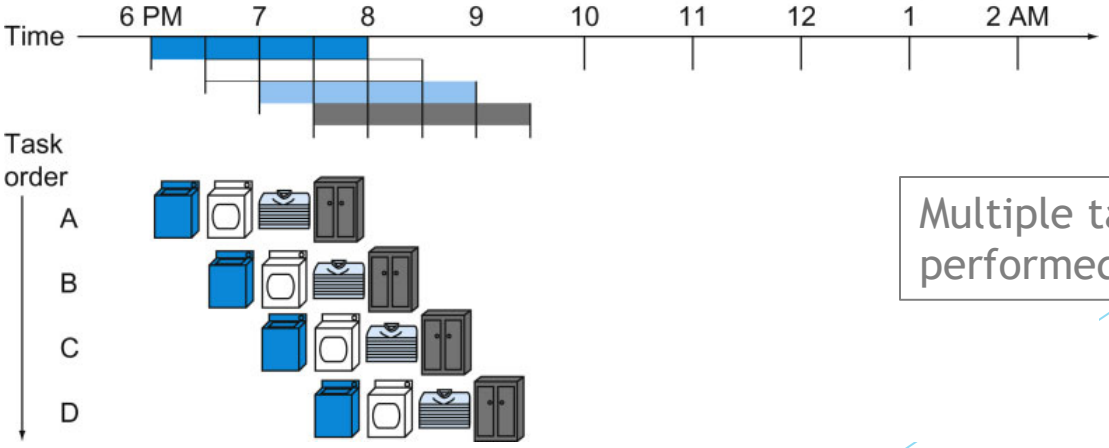
- Laundry Cycle



- Divided into 4 tasks.
- Each requires 30 minutes.



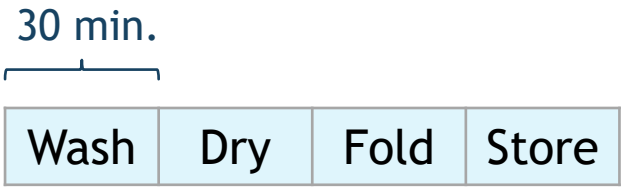
Only one task is performed at a time.



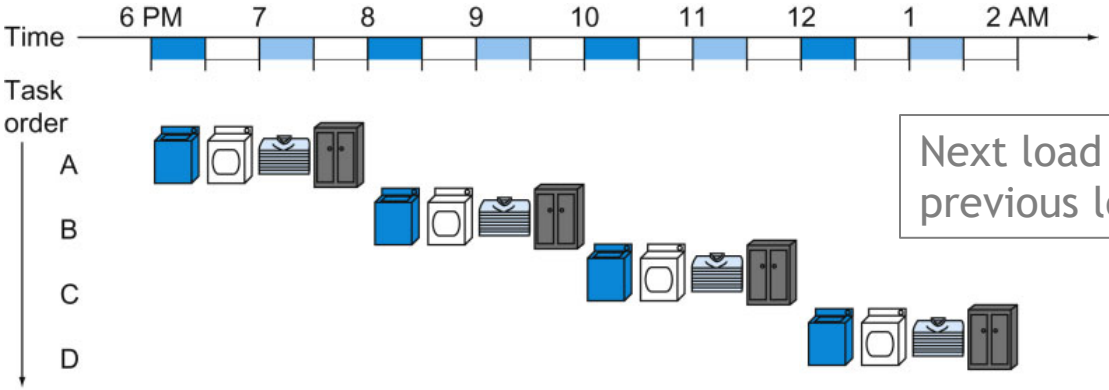
Multiple tasks can be performed at the same time.

Analogy

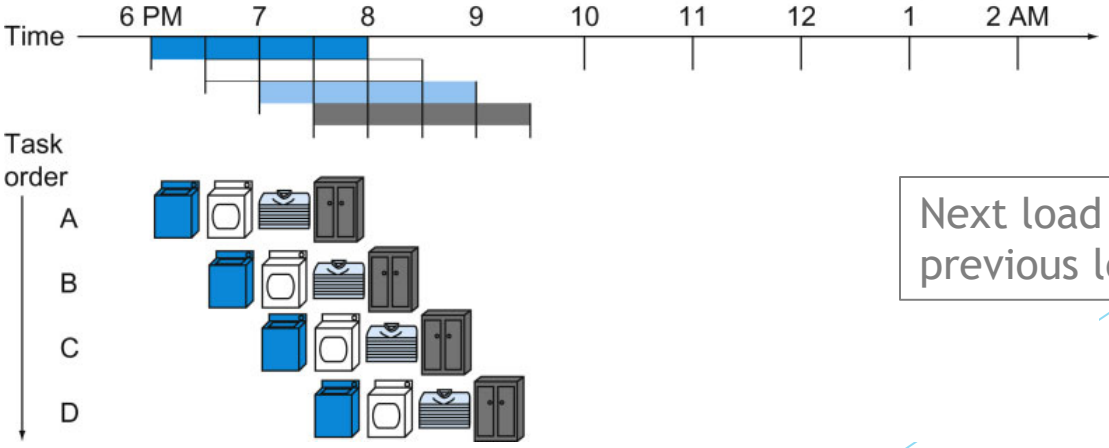
- Laundry Cycle



- Divided into 4 tasks.
- Each requires 30 minutes.



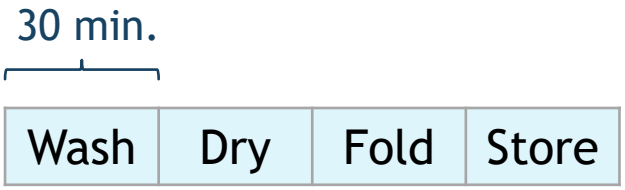
Next load started only after previous load completed.



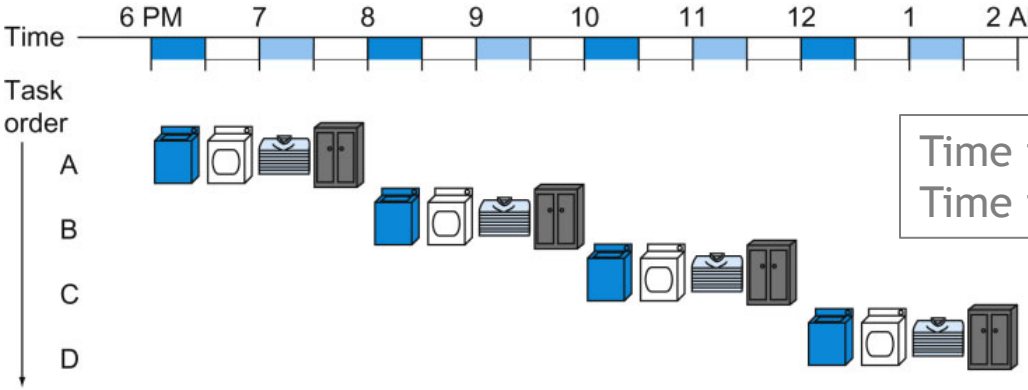
Next load started as soon as previous load washed.

Analogy

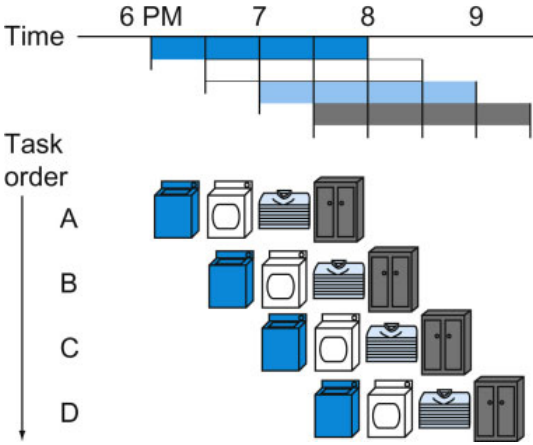
- Laundry Cycle



- Divided into 4 tasks.
- Each requires 30 minutes.



Time for one load = 120 minutes.
Time for four loads = 480 minutes.



Time for one load = 120 minutes.
Time for four loads = 210 minutes.

Each load requires 120 minutes.
Improvement when multiple loads processed.

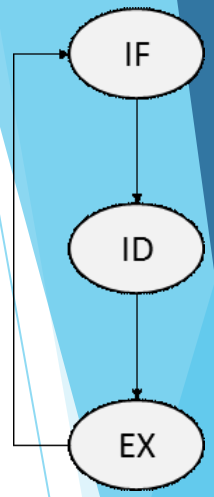
Instruction Pipelining

Instruction Cycle

Simplified three-stage Pipeline (IF, ID, EX)

Pipeline performance and overhead

Instruction Cycle



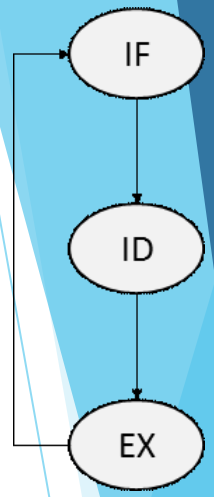
- The *instruction cycle* is divided into three steps.
- For a ***non-pipelined*** processor:

time →

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Instruction #1	IF	ID	EX												
Instruction #2				IF	ID	EX									
Instruction #3							IF	ID	EX						
Instruction #4										IF	ID	EX			
Instruction #5													IF	ID	EX

- One instruction is *fetched*, *decoded* and *executed* at a time.
- Next instruction is fetched only after previous instruction is completed.
- **Problem:** datapath resources are underutilized.

Instruction Cycle



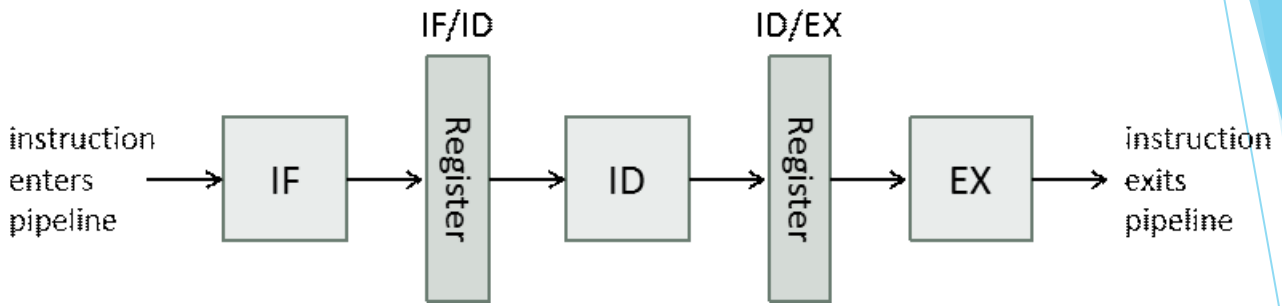
- The *instruction cycle* is divided into three steps.
- For a ***pipelined*** processor:

time →

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Instruction #1	IF	ID	EX												
Instruction #2		IF	ID	EX											
Instruction #3			IF	ID	EX										
Instruction #4				IF	ID	EX									
Instruction #5					IF	ID	EX								

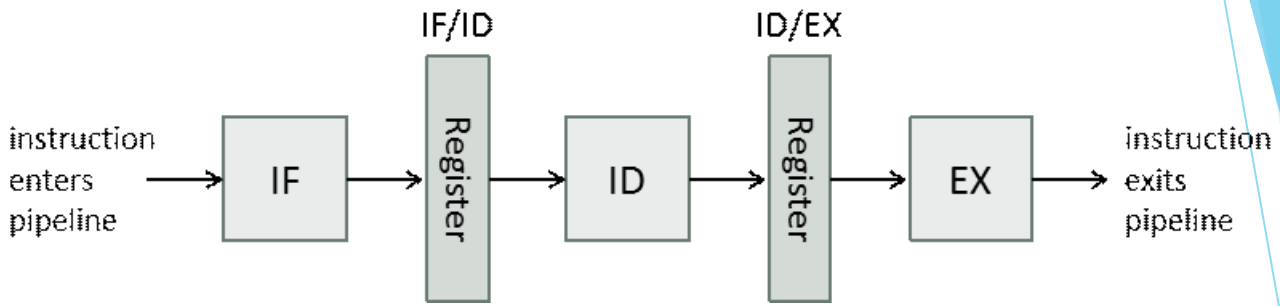
- Multiple instructions are processed at the same time.
- Next instruction is fetched when previous instruction is decoded.
- Each instruction is at a different step in the instruction cycle.

Simplified Three-stage Pipeline



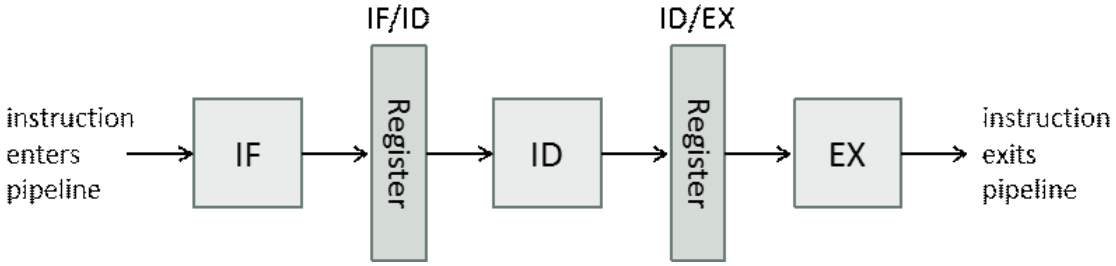
- Divide instruction processing into steps (or stages).
- The *simplified three-stage pipeline* has one stage for each step in the instruction cycle.
- Objective: keep all stages of the pipeline busy at all times.
  - Maximizes the processor performance.
- Pipelining improves processor performance by increasing **throughput** rather than decreasing **latency**.

Simplified Three-stage Pipeline



- Stages of the pipeline are separated by ***pipeline registers***.
- Pipeline registers provide independence between stages.
  - Each pipeline stage can operate independently of the other stages.
- At the end of a clock cycle, the output of one stage of the pipeline is stored in the corresponding pipeline register.
- This pipeline register then serves as the input for the next stage of the pipeline during the next clock cycle.

Simplified Three-stage Pipeline

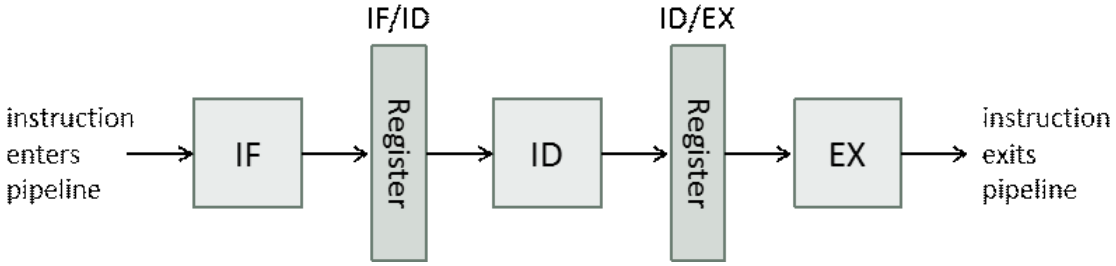


time

	IF	ID	EX
1	I1		
2	I2	I1	
3	I3	I2	I1
4	I4	I3	I2
5	I5	I4	I3
6	I6	I5	I4
7	I7	I6	I5
8		I7	I6
9			I7

- Each stage completed in one clock cycle.
  - Length of clock cycle determined by longest pipeline stage.
- Each instruction passes through all stages of the pipeline.
  - Even if no operation is performed.
- Each instruction completed in 3 clock cycles.
- Pipeline does not reduce **latency**.
  - Latency = time to complete instruction.

Simplified Three-stage Pipeline

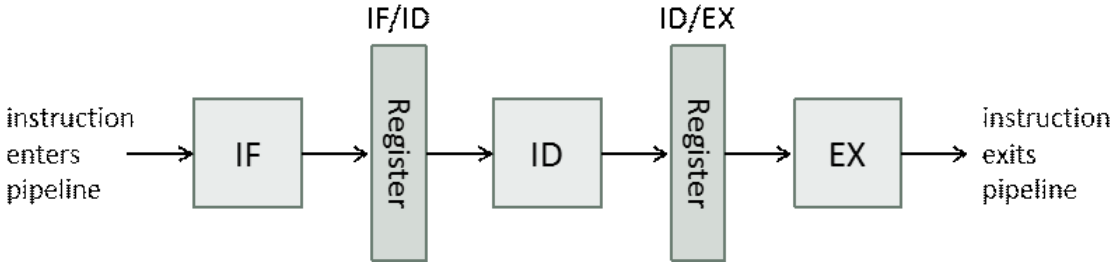


time

	IF	ID	EX
1	I1		
2	I2	I1	
3	I3	I2	I1
4	I4	I3	I2
5	I5	I4	I3
6	I6	I5	I4
7	I7	I6	I5
8		I7	I6
9			I7

- During any one clock cycle, each stage is processing a different instruction.
- Multiple instructions are processed during each clock cycle.
- Pipeline increases **throughput**.
  - Throughput = # instructions completed per time.
- Pipelining improves performance by increasing throughput.

Simplified Three-stage Pipeline



time

	IF	ID	EX
1	I1		
2	I2	I1	
3	I3	I2	I1
4	I4	I3	I2
5	I5	I4	I3
6	I6	I5	I4
7	I7	I6	I5
8		I7	I6
9			I7

- Ideally, the pipelined processor completes one instruction per clock cycle.
  - Throughput = 1 instruction per clock cycle.
- Pipeline must be *full*.
  - All stages must be processing an instruction.
  - Performance decreases if pipeline not full.
- Pipeline must be *balanced*.
  - Each stage must require the same amount of time.
  - Performance decreases if pipeline not balanced.

Pipeline Performance

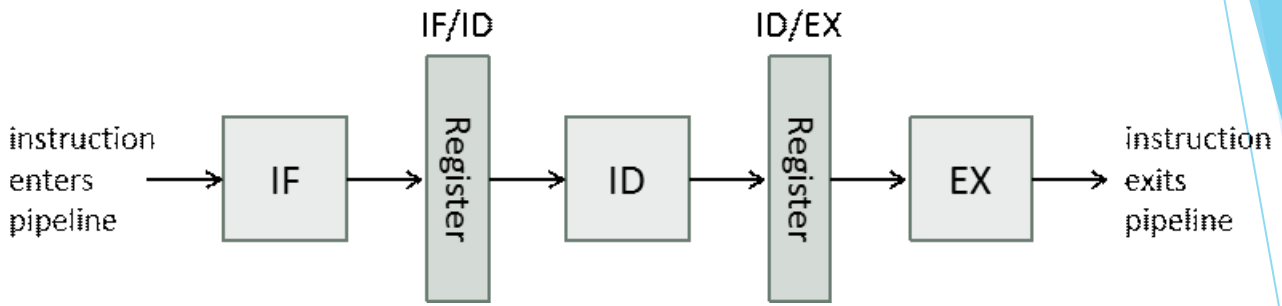
- Pipelining improves performance by increasing instruction throughput, not decreasing instruction latency.
  - Instruction latency = time to complete an instruction.
  - Instruction throughput = # of instructions completed per unit time.

- If stages of the pipeline are *balanced*, speedup is given by

$$\text{Time between instructions}_{\text{PIPELINE}} = \frac{\text{Time between instructions}_{\text{NON-PIPELINE}}}{\text{\# of Pipeline Stages}}$$

- If stages of the pipeline are *not balanced*, speedup is less.
  - Stages that require less time are idle for a portion of the clock cycle.
  - When stages of pipeline are idle, processor performance decreases.

Pipeline Overhead



- Pipeline registers contribute to the overhead of the pipeline.
  - Increase hardware (and cost) requirements of the processor.
  - Increase the instruction latency.
    - Clock-to-Q propagation delay.
    - Setup time
    - Hold time

    Timing characteristics of the pipeline registers.
- If the stages of the pipeline are not properly balanced.
  - Clock cycle time determined by longest pipeline stage.
  - Increases the instruction latency.

Pipelined MIPS

MIPS Organizations (Single-cycle, Multi-cycle, Pipelined)

MIPS 5-stage pipeline

Datapath components; Control unit(s)

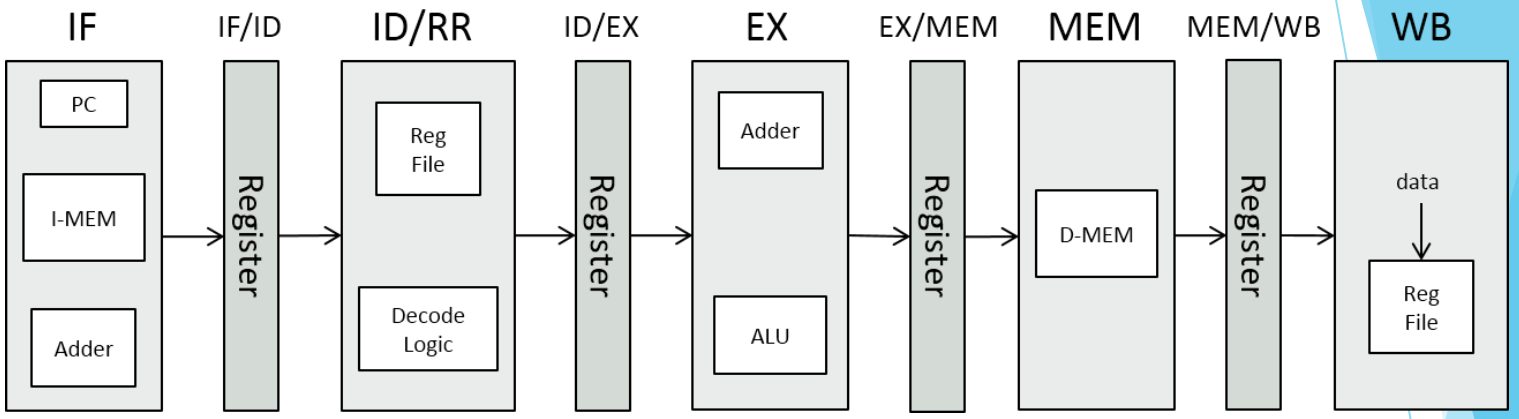
MIPS Pipeline

- Instruction processing is divided into steps (or stages).
- Each step (or stage) requires *one* clock cycle.
- The length of the clock cycle is determined by the time required to complete the longest step in the instruction processing.
- Overlap the execution of multiple instructions, such that each instruction is at a different stage of processing.
- All instructions must pass through every stage of the pipeline.
- All instructions require the same amount of time.

MIPS Pipeline

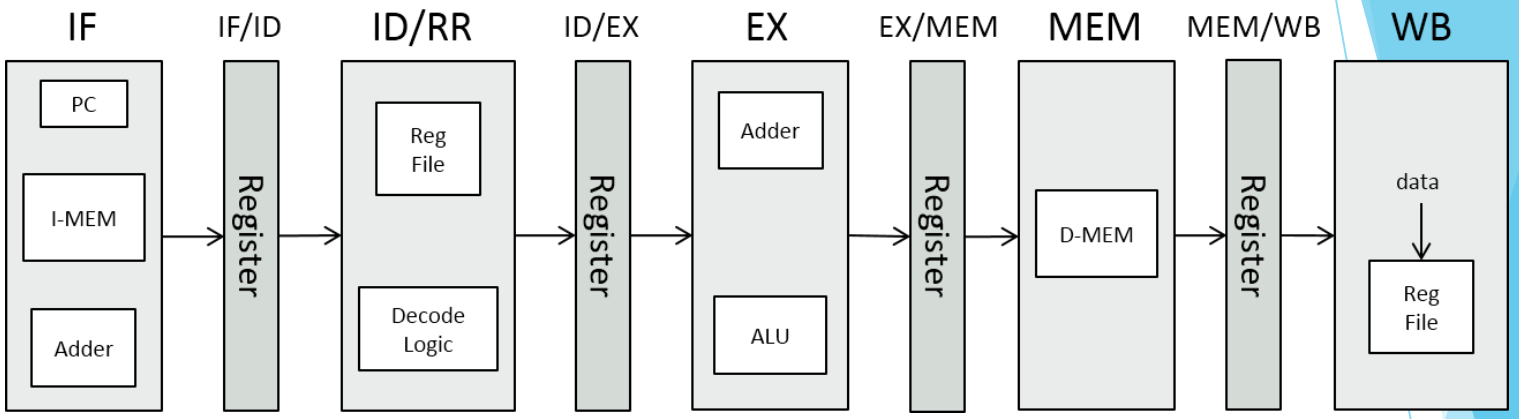
- MIPS instruction set architecture was designed for pipelining.
- All instructions are 32-bits.
  - Easier to fetch and decode in one clock cycle (each).
  - As compared to x86 processors which have 1 – 17 byte instructions.
- Few and regular instruction formats
  - Can decode and read registers in same step (or stage).
- Memory addressing (load and store)
  - Calculate memory address in one stage.
  - Access memory in the next stage.
- Memory operands aligned on word boundaries
  - Memory access takes one clock cycle.

MIPS Pipeline



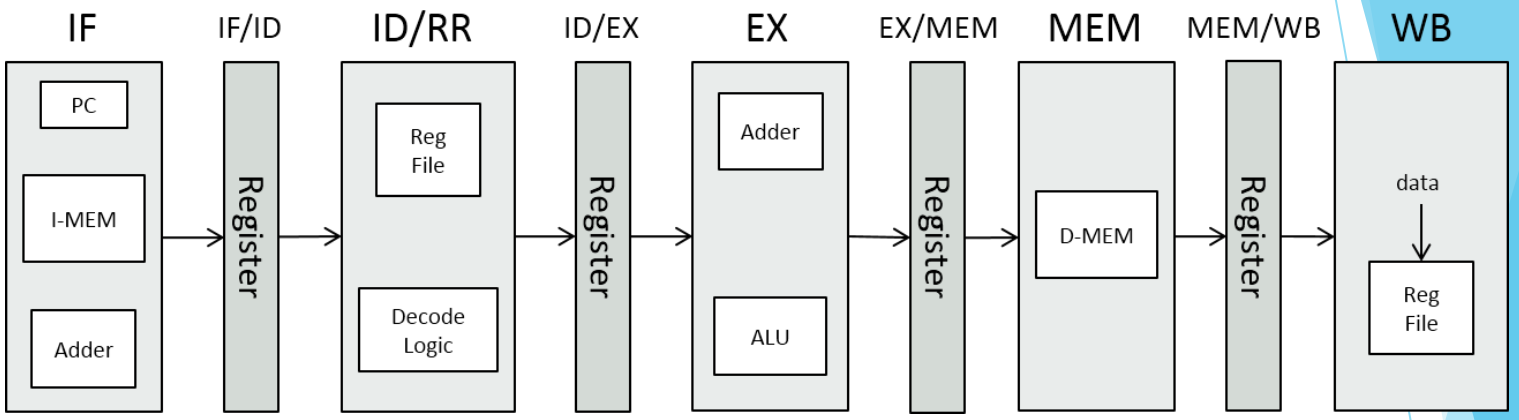
- MIPS pipeline divided into ***five*** stages.
  1. IF: Instruction fetch from memory
  2. ID: Instruction decode & register read
  3. EX: Execute operation or calculate address
  4. MEM: Access memory operand
  5. WB: Write result back to register

MIPS Pipeline



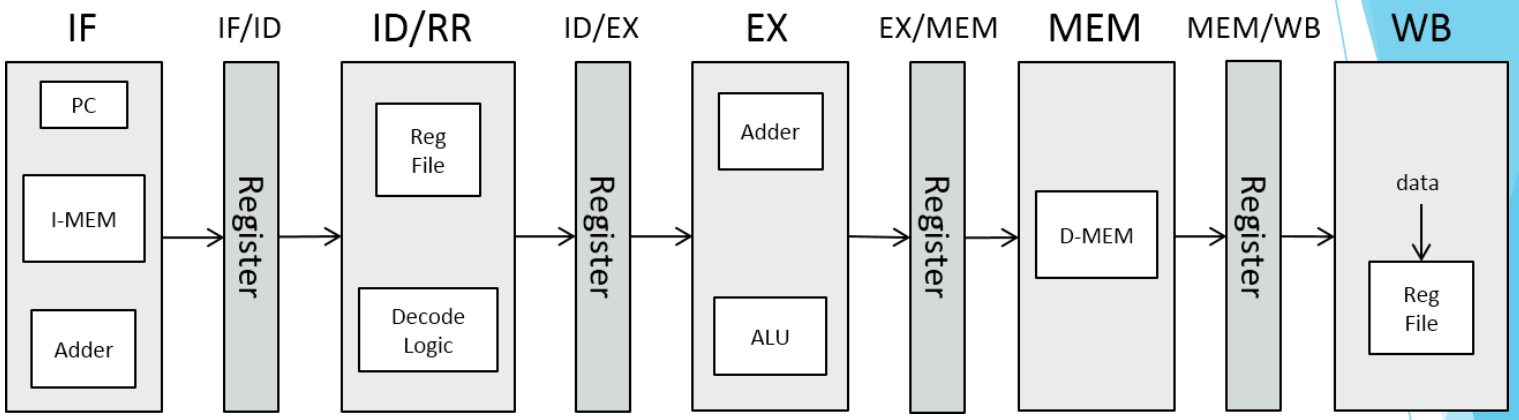
- MIPS pipeline divided into ***five*** stages.
- Stages of the pipeline are well-balanced.
  - Pipeline stages require approximately the same amount of time.
  - Leads to a performance improvement that approaches $N_{IMPROVE} = 5$.
- Pipeline registers separate the pipeline stages.
- Duplicate logic needed to avoid resource contention.

MIPS Pipeline



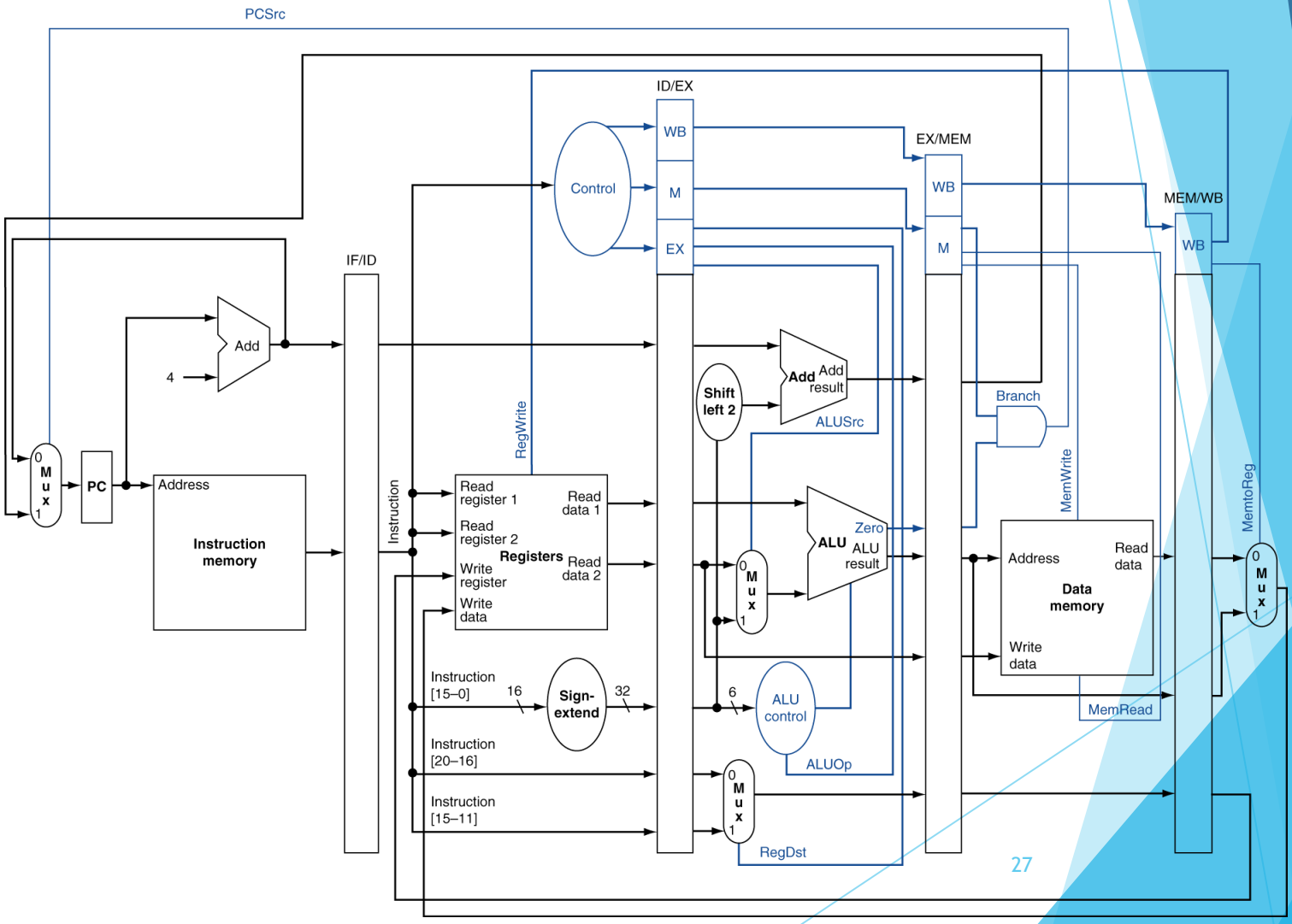
- There are five instructions in the pipeline at steady-state.
  - One instruction is in each stage of the pipeline.
- Every instruction requires five clock cycles to process.
  - One clock cycle for each stage of the pipeline.
  - Pipelining does not decrease the instruction **latency**.
  - Pipelining increases the **throughput** of the processor.

MIPS Pipeline



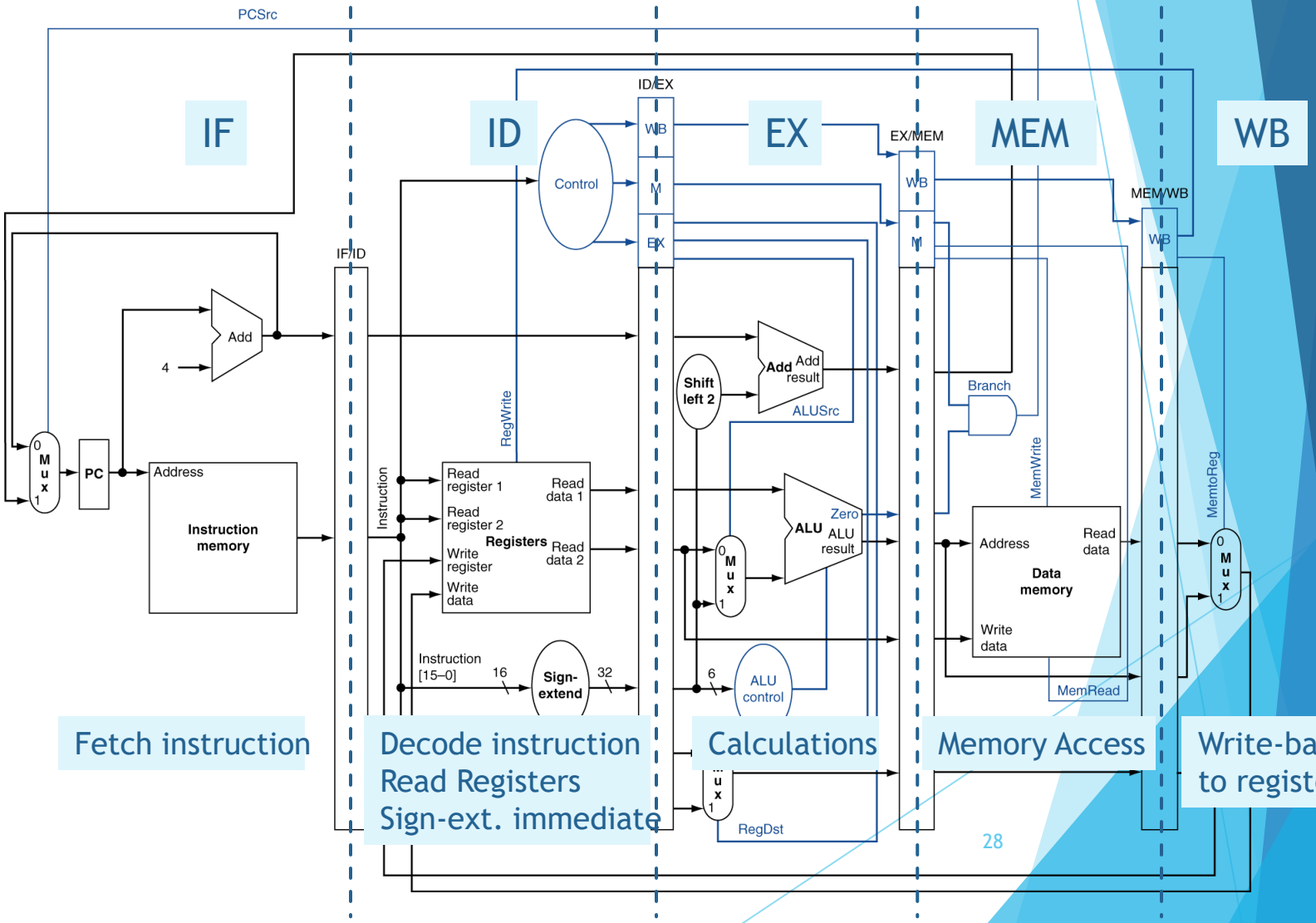
- Every instruction
  - Enters the pipeline at the IF (instruction fetch) stage.
  - Exits the pipeline at the WB (write back) stage.
- Ideally, the processor completes one instruction per clock cycle.
  - Assuming that the pipeline is full.
  - IPC_{IDEAL} = instructions per clock cycle = 1.
  - CPI_{IDEAL} = clock cycles per instruction = 1 for the pipelined processor.

The Pipelined MIPS Processor

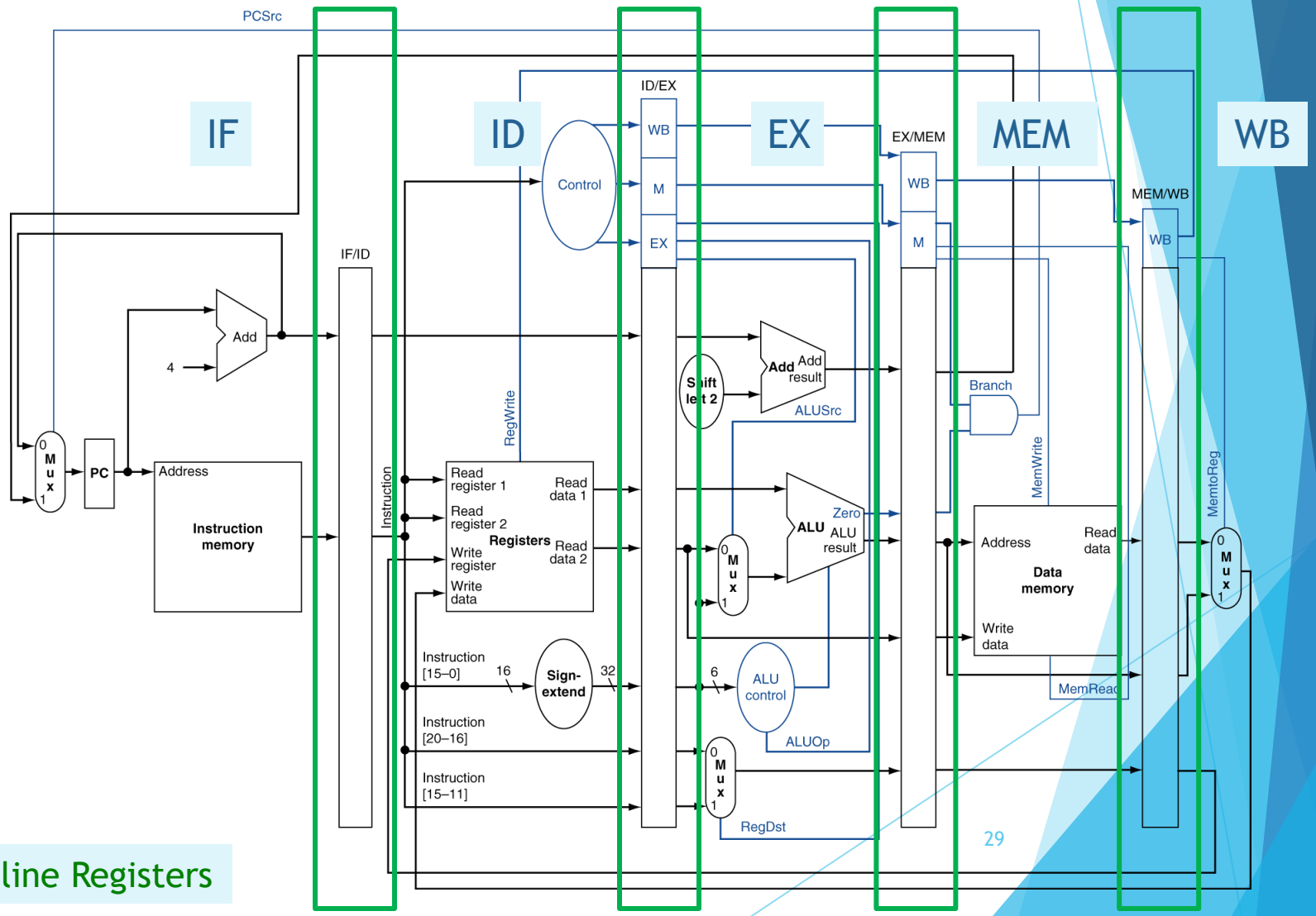


"Computer Organization and Design", 5th Edition; Hennessey and Patterson; p. 304

The Pipelined MIPS Processor



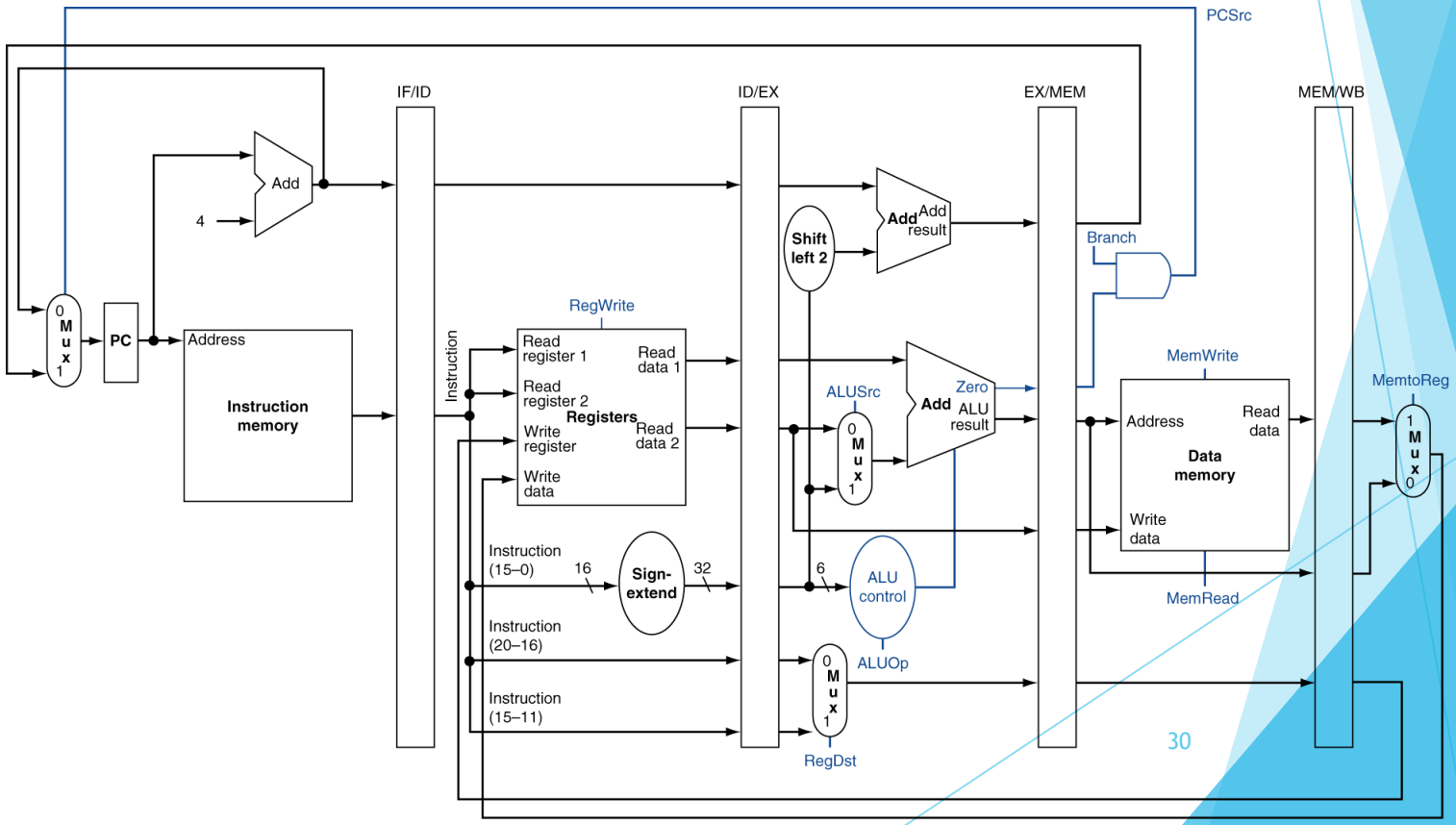
The Pipelined MIPS Processor



Pipeline Registers

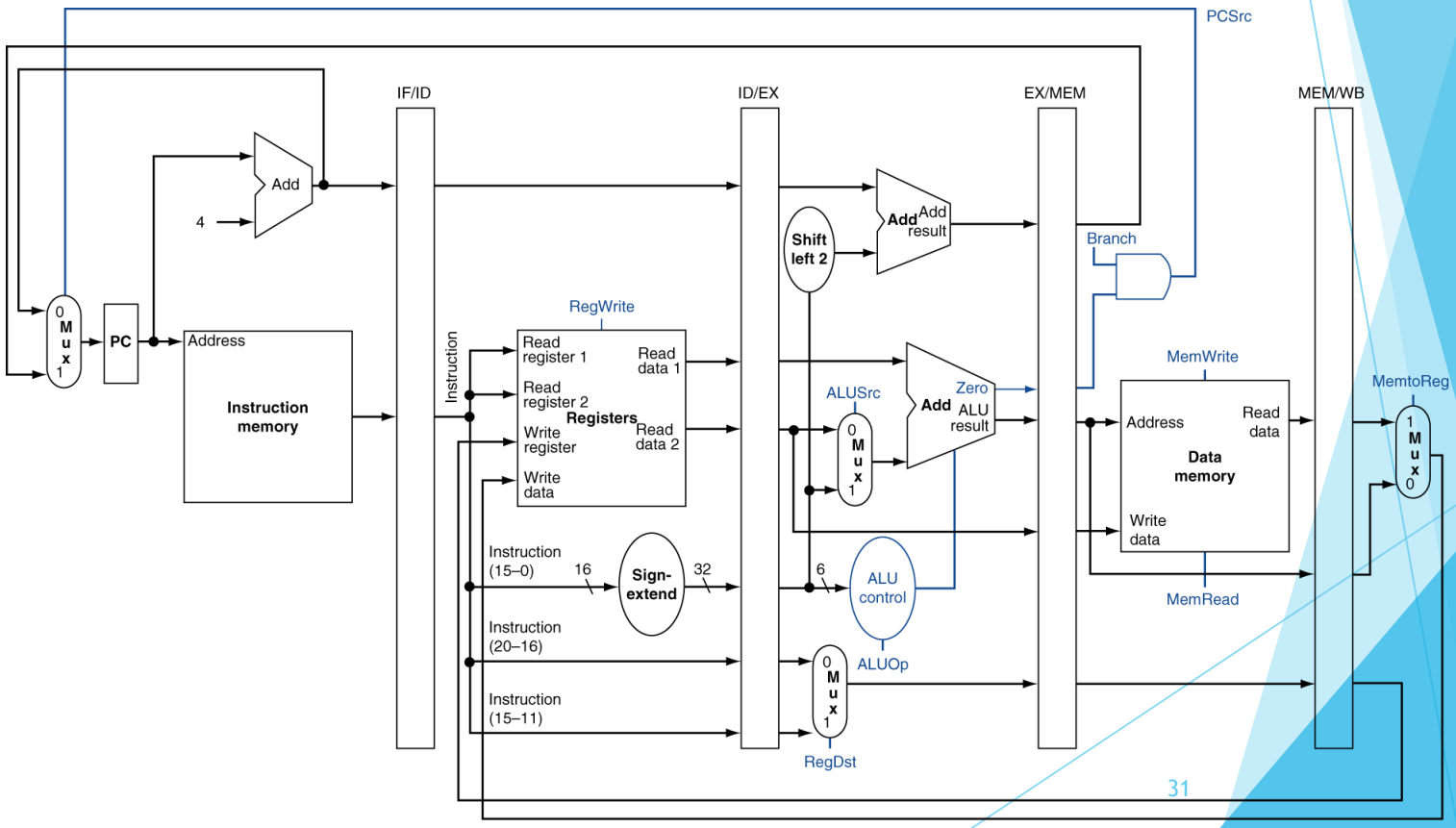
Pipeline: Datapath Components

- Each pipeline stage contains the needed datapath components.
- Where necessary, datapath components are duplicated.

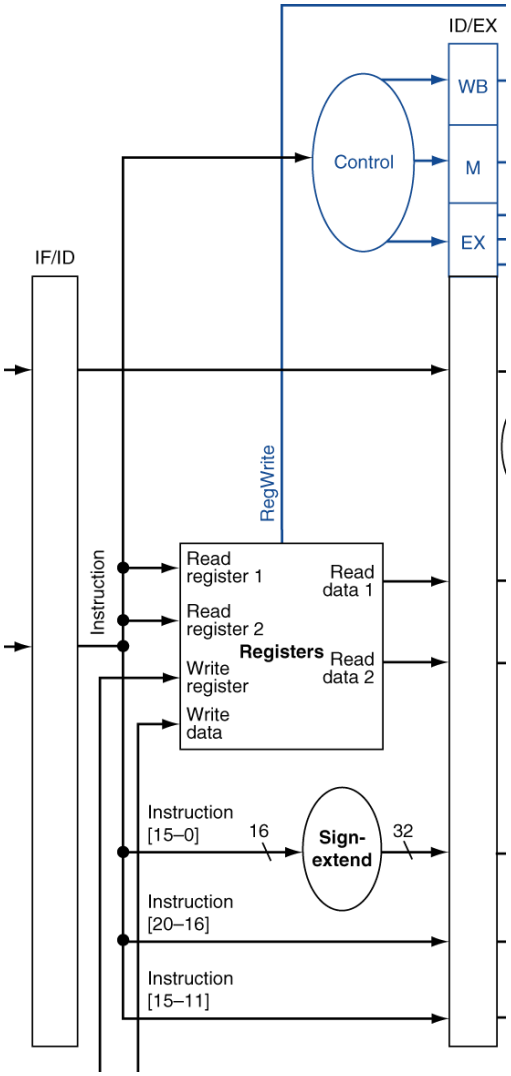


Pipeline: Control Signals

- Requires the same control signals as the *single-cycle* MIPS.



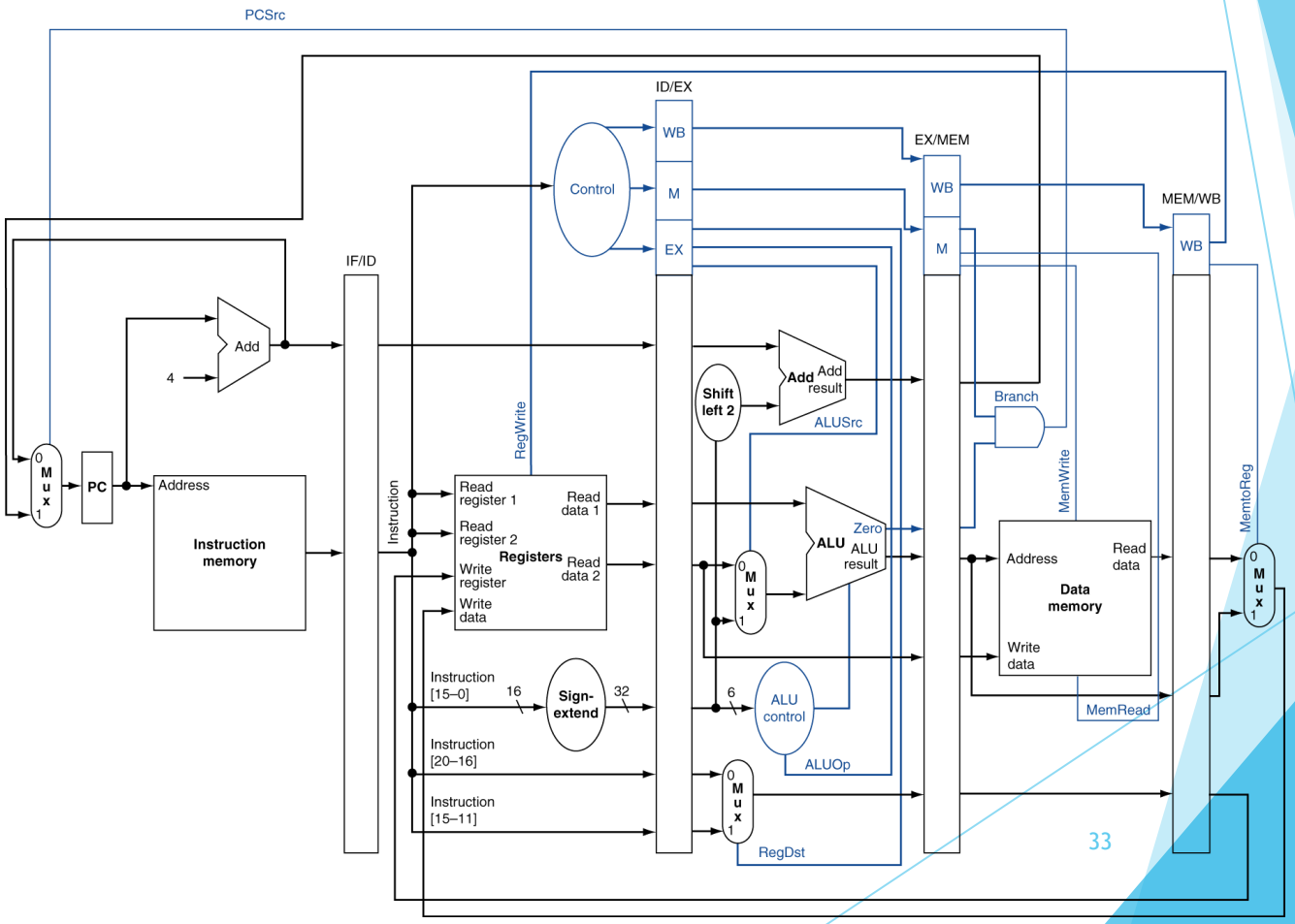
Pipeline: Control Unit



- Control unit is part of the ID stage of the pipeline.
- Control unit is combinational logic.
- Control signals are derived from the instruction.
  - Opcode
  - Funct bits (R-type)

Pipeline: Control Signals

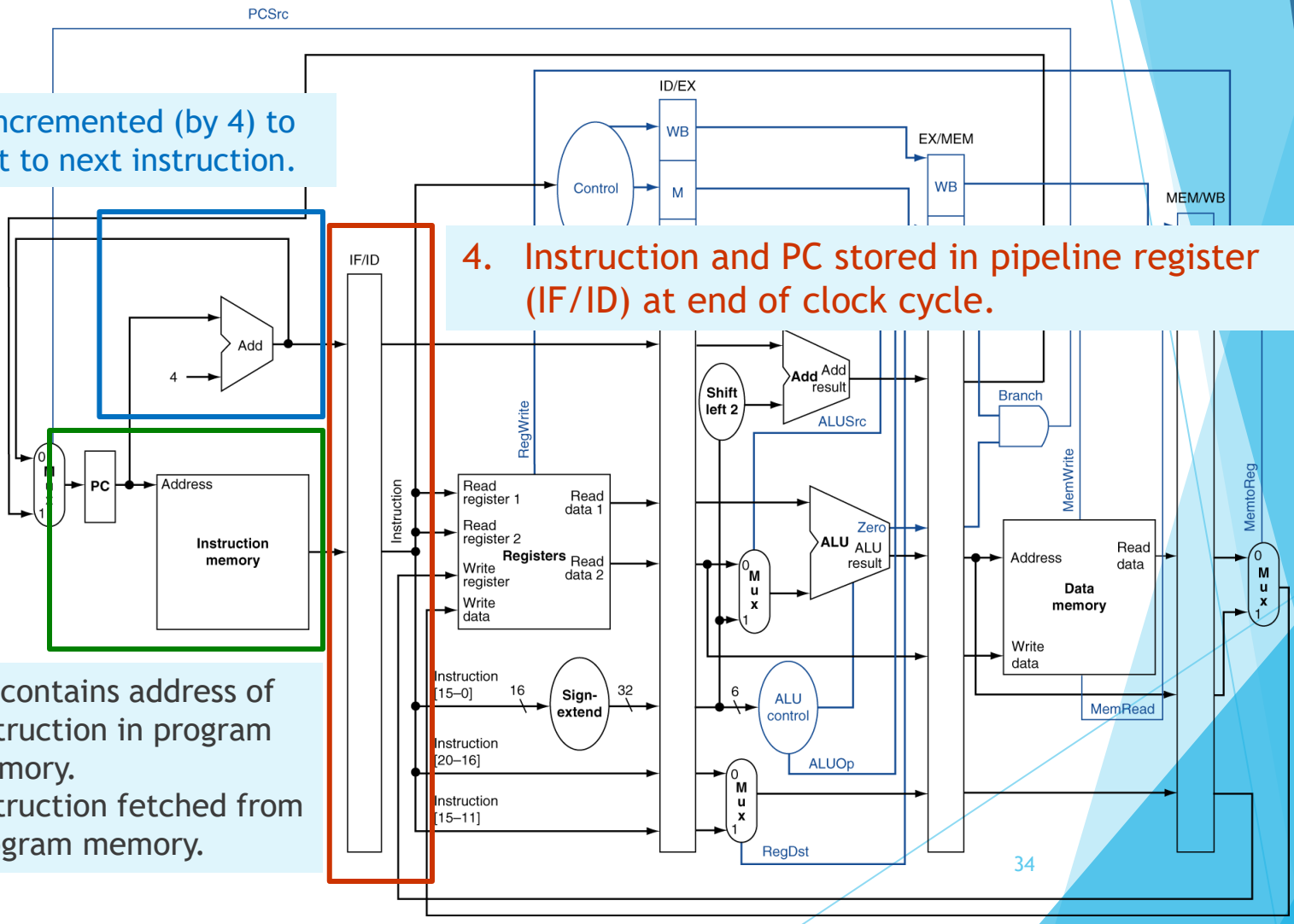
- Control signals propagate through pipeline with instruction.



Pipelined MIPS (IF Stage)

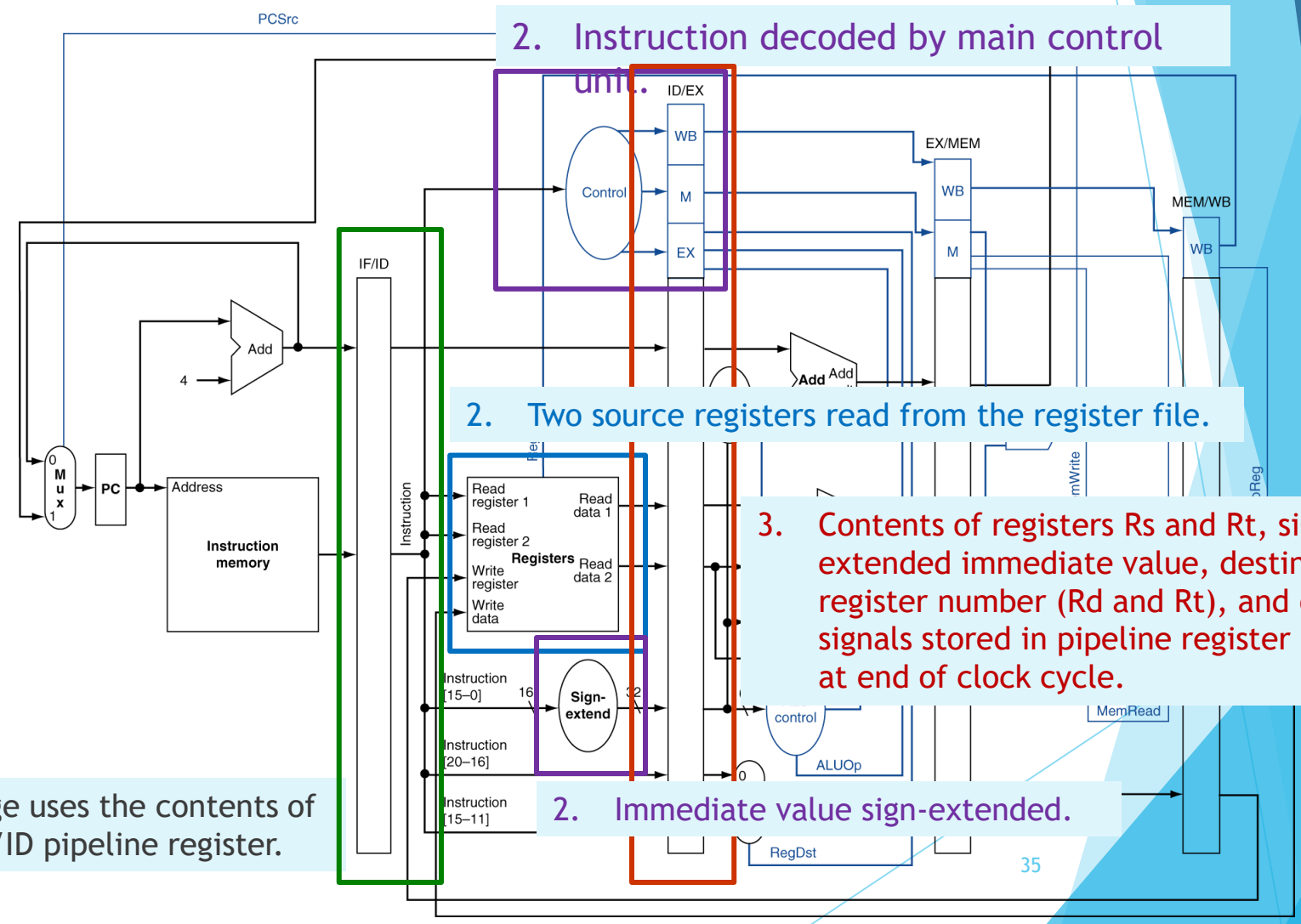
3. PC incremented (by 4) to point to next instruction.

4. Instruction and PC stored in pipeline register (IF/ID) at end of clock cycle.



1. PC contains address of instruction in program memory.
2. Instruction fetched from program memory.

Pipelined MIPS (ID Stage)



2. Instruction decoded by main control unit.

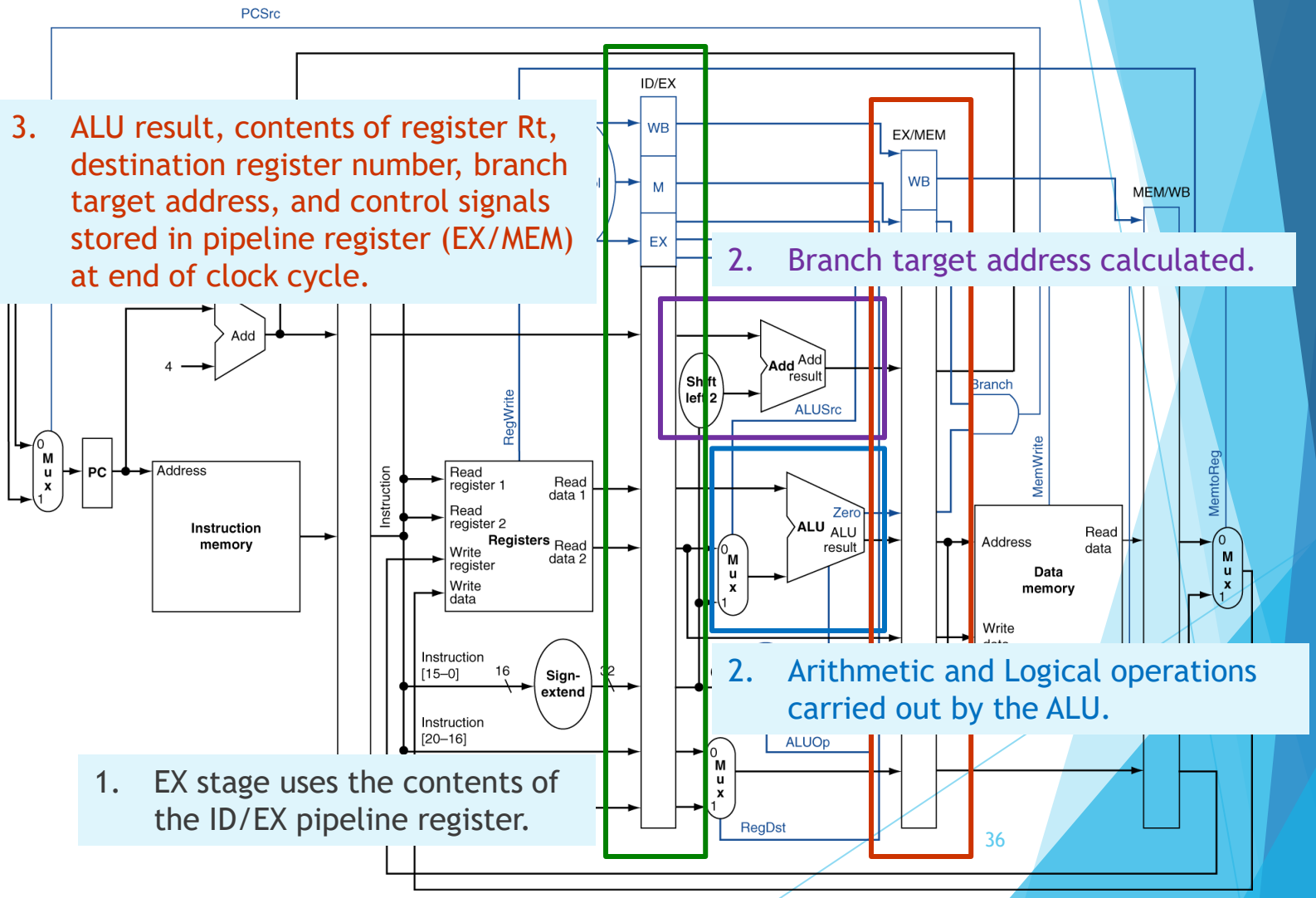
2. Two source registers read from the register file.

3. Contents of registers Rs and Rt, sign-extended immediate value, destination register number (Rd and Rt), and control signals stored in pipeline register (ID/EX) at end of clock cycle.

1. ID stage uses the contents of the IF/ID pipeline register.

2. Immediate value sign-extended.

Pipelined MIPS (EX Stage)



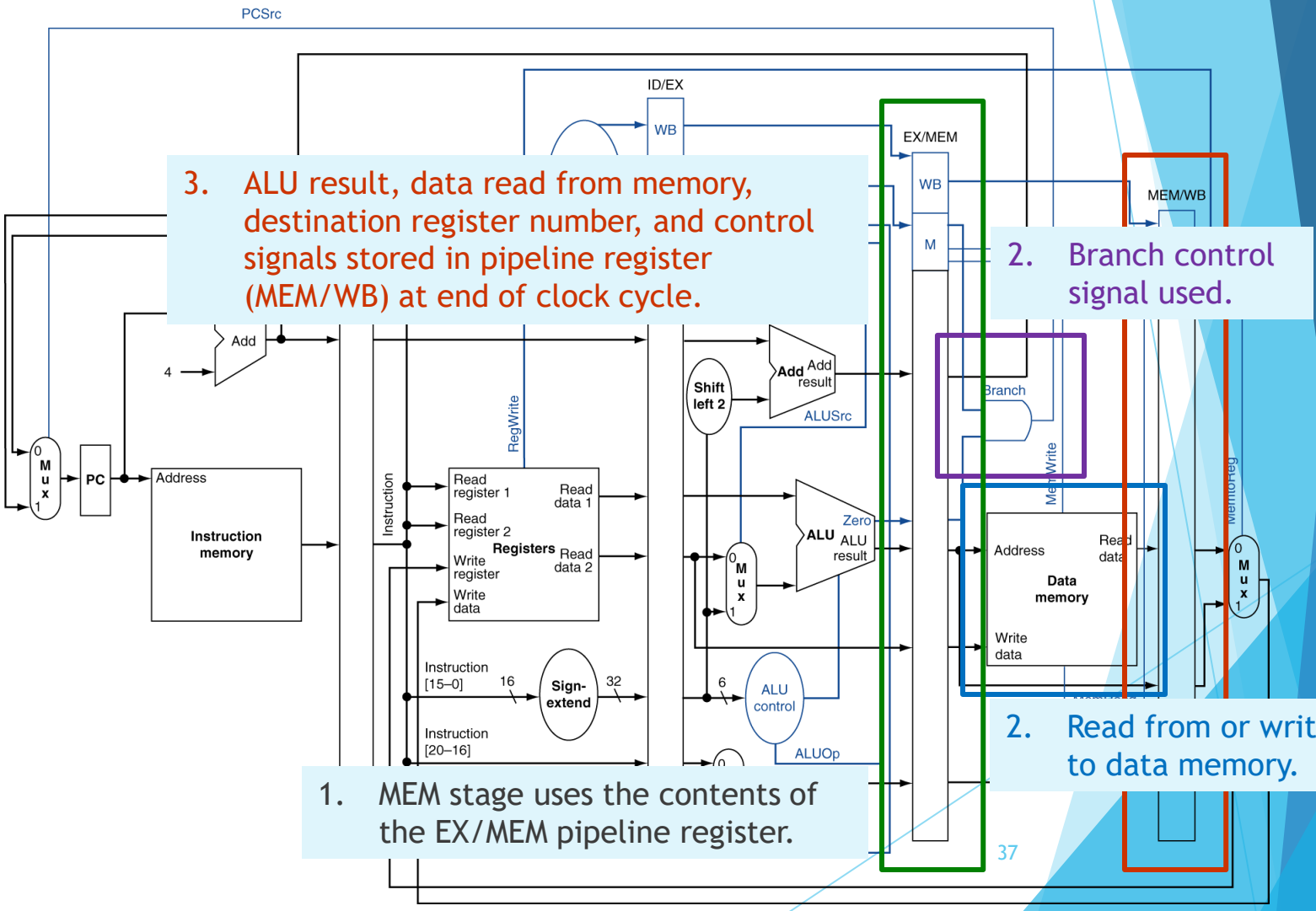
3. ALU result, contents of register Rt, destination register number, branch target address, and control signals stored in pipeline register (EX/MEM) at end of clock cycle.

2. Branch target address calculated.

2. Arithmetic and Logical operations carried out by the ALU.

1. EX stage uses the contents of the ID/EX pipeline register.

Pipelined MIPS (MEM Stage)



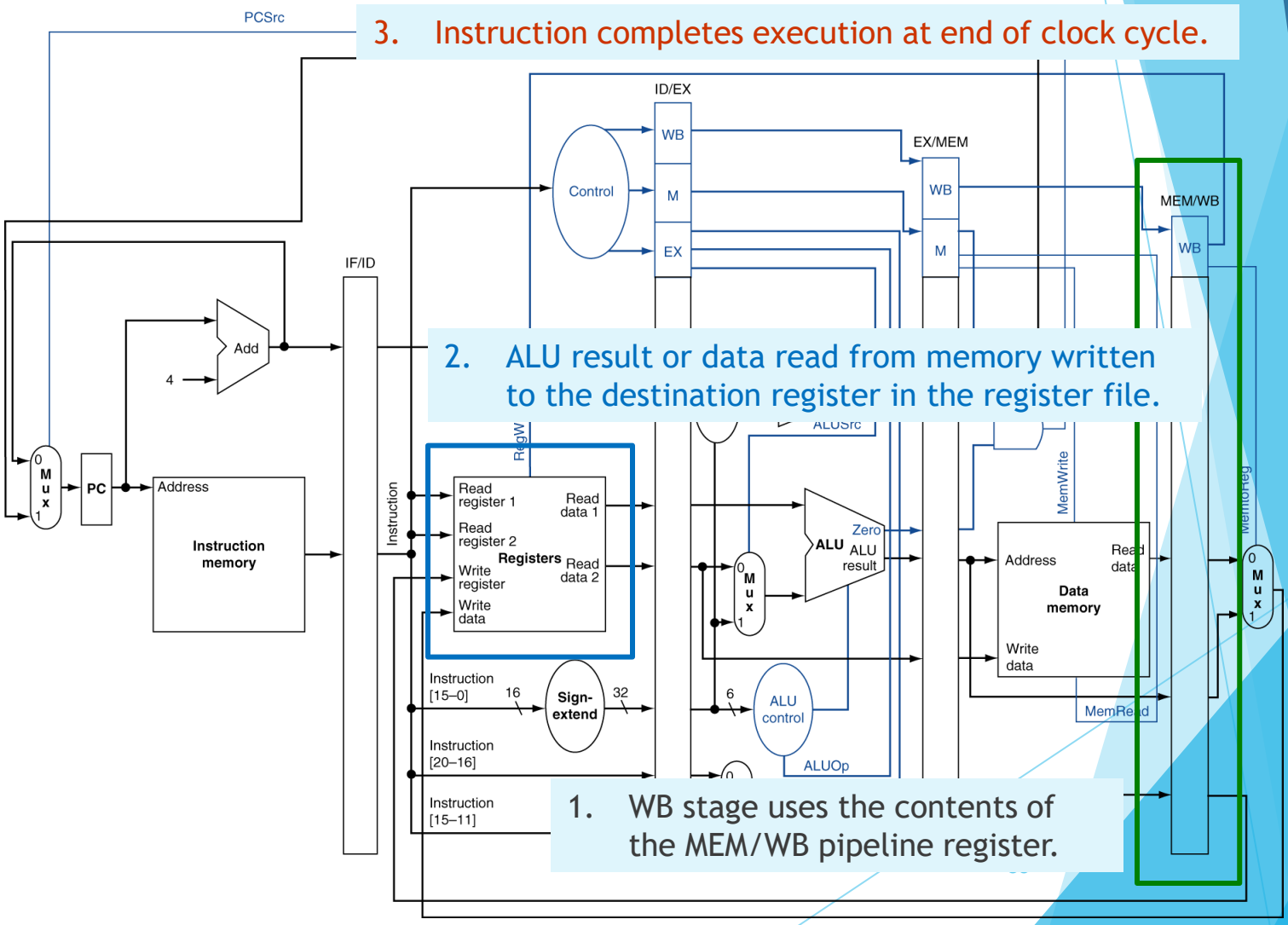
3. ALU result, data read from memory, destination register number, and control signals stored in pipeline register (MEM/WB) at end of clock cycle.

2. Branch control signal used.

2. Read from or write to data memory.

1. MEM stage uses the contents of the EX/MEM pipeline register.

Pipelined MIPS (WB Stage)

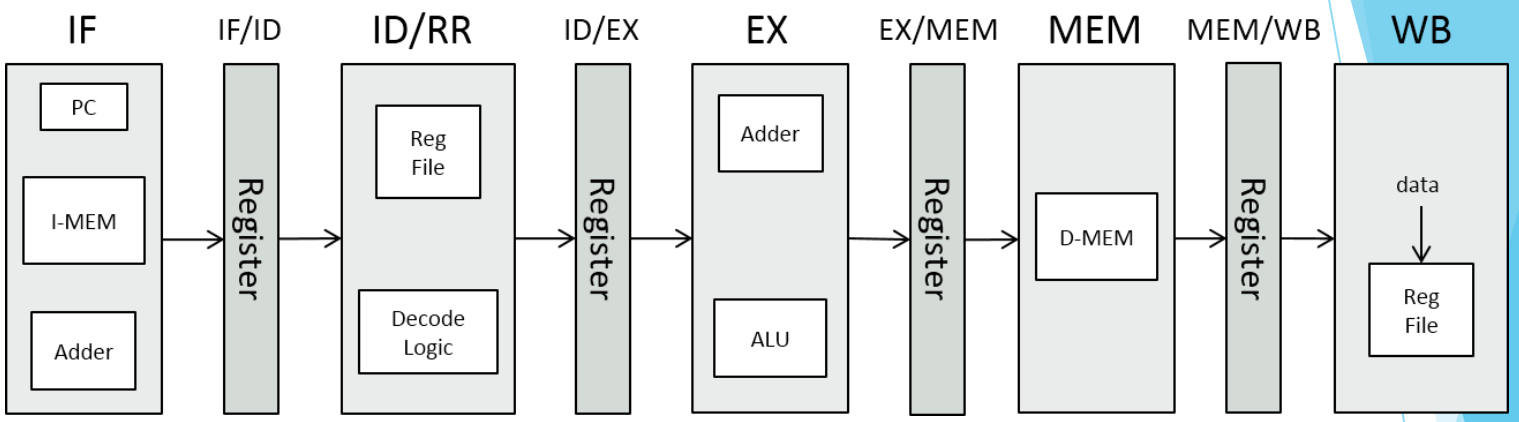


3. Instruction completes execution at end of clock cycle.

2. ALU result or data read from memory written to the destination register in the register file.

1. WB stage uses the contents of the MEM/WB pipeline register.

MIPS Pipeline and Memory



- Processor performance is dependent on the memory system.
- Memory access time is dominant latency in pipelined processor.
  - Fetching instructions from I-MEM (IF stage).
  - Reading from and writing to D-MEM (MEM stage).
  - Processor hides this latency by using ***caches***.
- Accessing memory takes multiple clock cycles.
  - Processor must be ***stalled*** while waiting for memory.
  - Continues processing instructions after memory access completed.

Pipelined MIPS

Processing instructions in the MIPS 5-stage Pipeline.

Instruction Processing

- The MIPS 5-stage pipeline processes instructions as follows:

```
I1:   lw   $t0, 20($s1)
I2:   sub  $t1, $s2, $s3
I3:   add  $t2, $s3, $s4
I4:   lw   $t3, 24($s1)
I5:   add  $t4, $s5, $s6
I6:   addi $t5, $t6, 1
```

Clock Cycle #0

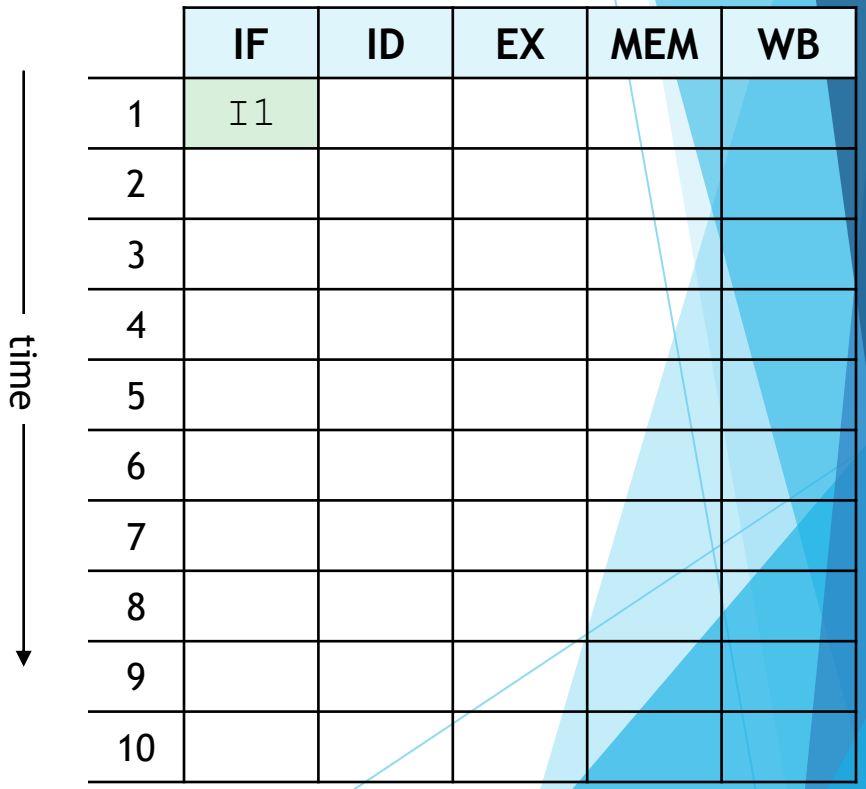
time

	IF	ID	EX	MEM	WB
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					

Instruction Processing

- The MIPS 5-stage pipeline processes instructions as follows:

```
I1:  lw   $t0, 20($s1)
I2:  sub  $t1, $s2, $s3
I3:  add  $t2, $s3, $s4
I4:  lw   $t3, 24($s1)
I5:  add  $t4, $s5, $s6
I6:  addi $t5, $t6, 1
```



Clock Cycle #1

Instruction I1 enters the pipeline.

Instruction Processing

- The MIPS 5-stage pipeline processes instructions as follows:

```
I1:   lw   $t0, 20($s1)
I2:   sub  $t1, $s2, $s3
I3:   add  $t2, $s3, $s4
I4:   lw   $t3, 24($s1)
I5:   add  $t4, $s5, $s6
I6:   addi $t5, $t6, 1
```

Clock Cycle #2

time

	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3					
4					
5					
6					
7					
8					
9					
10					

Instruction Processing

- The MIPS 5-stage pipeline processes instructions as follows:

```
I1:   lw   $t0, 20($s1)
I2:   sub  $t1, $s2, $s3
I3:   add  $t2, $s3, $s4
I4:   lw   $t3, 24($s1)
I5:   add  $t4, $s5, $s6
I6:   addi $t5, $t6, 1
```

Clock Cycle #3

time

	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3	I3	I2	I1		
4					
5					
6					
7					
8					
9					
10					

Instruction Processing

- The MIPS 5-stage pipeline processes instructions as follows:

```
I1:   lw   $t0, 20($s1)
I2:   sub  $t1, $s2, $s3
I3:   add  $t2, $s3, $s4
I4:   lw   $t3, 24($s1)
I5:   add  $t4, $s5, $s6
I6:   addi $t5, $t6, 1
```

Clock Cycle #4

time

	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3	I3	I2	I1		
4	I4	I3	I2	I1	
5					
6					
7					
8					
9					
10					

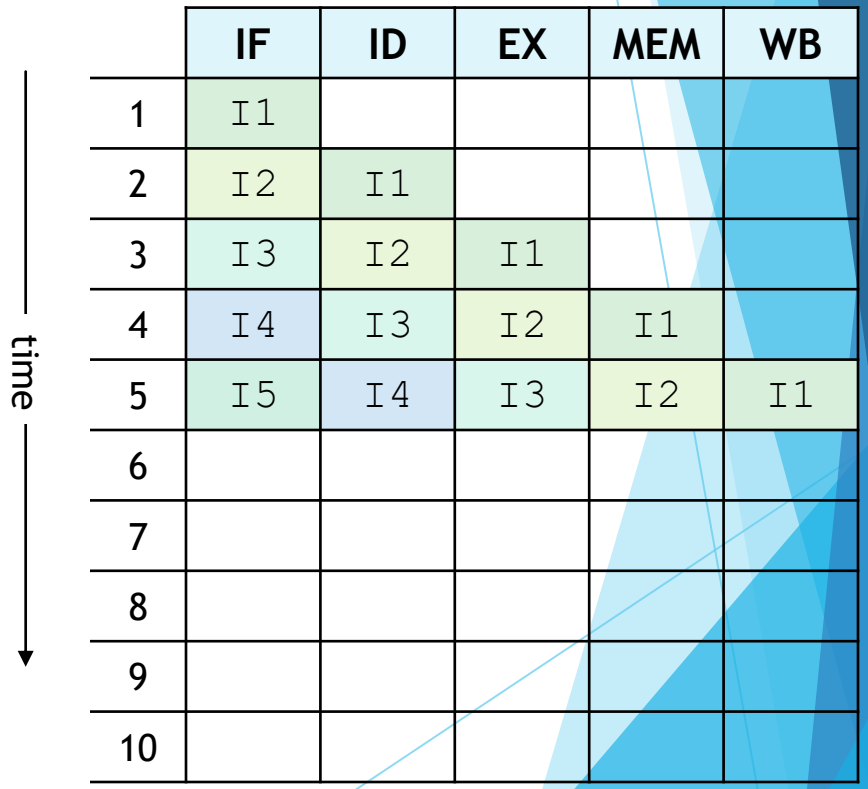
Instruction Processing

- The MIPS 5-stage pipeline processes instructions as follows:

```
I1:   lw   $t0, 20($s1)
I2:   sub  $t1, $s2, $s3
I3:   add  $t2, $s3, $s4
I4:   lw   $t3, 24($s1)
I5:   add  $t4, $s5, $s6
I6:   addi $t5, $t6, 1
```

Pipeline is *full*.

Clock Cycle #5



Instruction I1 exits pipeline at end of CC #5

Instruction Processing

- The MIPS 5-stage pipeline processes instructions as follows:

```
I1:   lw   $t0, 20($s1)
I2:   sub  $t1, $s2, $s3
I3:   add  $t2, $s3, $s4
I4:   lw   $t3, 24($s1)
I5:   add  $t4, $s5, $s6
I6:   addi $t5, $t6, 1
```

Pipeline is *full*.

Clock Cycle #6

time

	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3	I3	I2	I1		
4	I4	I3	I2	I1	
5	I5	I4	I3	I2	I1
6	I6	I5	I4	I3	I2
7					
8					
9					
10					

Instruction I2 exits pipeline at end of CC #6

Instruction Processing

- The MIPS 5-stage pipeline processes instructions as follows:

```
I1:   lw   $t0, 20($s1)
I2:   sub  $t1, $s2, $s3
I3:   add  $t2, $s3, $s4
I4:   lw   $t3, 24($s1)
I5:   add  $t4, $s5, $s6
I6:   addi $t5, $t6, 1
```

Clock Cycle #7

time

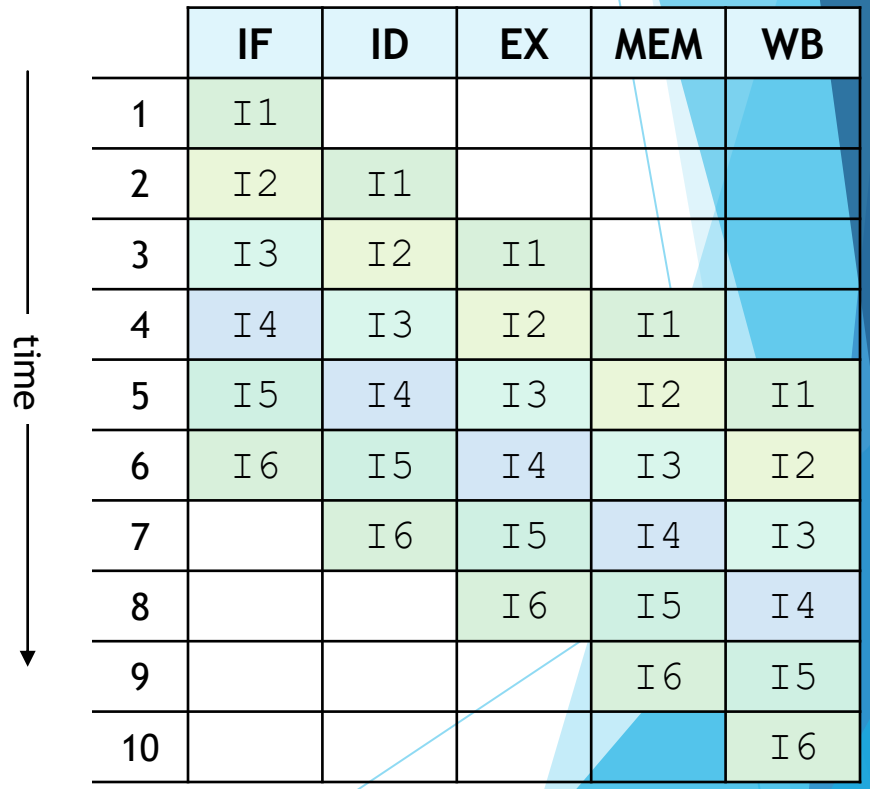
	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3	I3	I2	I1		
4	I4	I3	I2	I1	
5	I5	I4	I3	I2	I1
6	I6	I5	I4	I3	I2
7		I6	I5	I4	I3
8					
9					
10					

Instruction I3 exits pipeline at end of CC #7

Instruction Processing

- The MIPS 5-stage pipeline processes instructions as follows:

```
I1:   lw   $t0, 20($s1)
I2:   sub  $t1, $s2, $s3
I3:   add  $t2, $s3, $s4
I4:   lw   $t3, 24($s1)
I5:   add  $t4, $s5, $s6
I6:   addi $t5, $t6, 1
```



Clock Cycle #10

Instruction I6 exits pipeline at end of CC #10